

misc/bibliography.bib

Investigating graph stability in graphs derived from EEG data

Cristina Avram, Lili Raleva and Marius Thomsen

May 2025

IT UNIVERSITY OF COPENHAGEN

Contents

Abstract

Instability in the graph learned from DGCNN has been observed. To understand this phenomenon. We investigate the stability of graph representations learned by DGCNN models trained on a motor imagery task. To assess these representations and employ representational similarity using CKA, and examine the graph centrality metrics of the learned graph representations.

Our analysis reveals a consistent pattern: the electrodes C3 and CP3 repeatedly emerge as central nodes across all evaluated centrality measures. Given their anatomical locations within the motor and sensorimotor cortex, areas closely associated with motor planning and execution, this finding suggests that the models are capturing relevant features aligned with the underlying task. We, thus, do observe a notable degree of consistency in the learned graph representations across multiple runs, despite the different random initialisations and internal representations.

Code for repository: github.com/AlbinoLoaf/brAIncells

1 Introduction

We constantly have something on our minds – the neurons in our brains are turbulently exchanging information. This information is in the form of electrical signals, and scientists have long ago discovered a way of non-invasively recording this activity – electroencephalogram, referred to as EEG ([?]). EEG is a method of measuring continuous patterns of electrical activity in underlying regions of the brain ([?]). To record this activity, electrodes are positioned on different areas of the scalp.

Since EEG is significantly advantageous in multiple ways, it has acquired a wide use across multiple fields, some of which being psychophysiology, cognitive, and neuroscience. The analysis of EEG consists of combining mathematical methods and computer technology, whose synergy has led to researchers gaining a deeper understanding of the inner workings of the brain. Typically, EEG analysis methods are categorized as falling into the time, frequency, time-frequency, or non-linear domains. In addition, with the development of deep learning, a new category of analysis has emerged – namely, EEG research and analysis via deep artificial neural networks.

There are many kinds of neural networks, thus, many ways of analysing EEG signals. One of them, which has acquired a wide use among researchers and scientists, is the graph neural network (GNN). In general, GNNs have proved to be well-suited for dealing with EEG data, as they are able to model complex structures, which are inherent in brain connectivity ([?]). A specific type of a GNN, which was shown to be a great tool for EEG analysis and research, is the dynamic graph convolutional neural network (DGCNN). It can adaptively learn the innate relationships of EEG channels, allowing the algorithm to produce a more accurate graph representation of the EEG data ([?]). Graph representations, in turn, offer an intuitive way of modelling brain activity. In the case of working with EEG – representing electrodes as nodes and their interactions as edges allows for the capturing of patterns of functional connectivity between different brain regions. This structure-oriented approach aligns well with the brain’s inherently networked organization and opens the door to leveraging a

wide range of tools from network analysis. Via them, we may manage to reveal interpretable insights about the activity from and between functionally different brain areas.

However, for such insights to be meaningful, it is desirable, that the learned graphs are stable, meaning that their structures are consistent. If repeated training runs on the same data produce significantly different graph structures, it becomes difficult to draw conclusions and trust the model’s graph constructing ability. This instability can significantly undermine the reliability of the model and the produced results.

Despite these challenges, the potential of EEG analysis, carried out with the aid of artificial neural networks, remains compelling. Our research deals with investigating the consistency of graph structures learned by GNNs. Specifically, we explore methods for measuring representational similarity and identifying persistent patterns across training runs, aiming to better understand how reliable these learned graphs truly are.

Problem statement

This project investigates the stability of learned graph structures inferred from EEG data using GNNs. While GNNs have shown promising performance for EEG tasks, such as motor imagery classification, the structure of the learned graphs remains poorly understood and may vary between training runs.

We aim to uncover potential structural or functional consistencies in the learned graph representations by examining a range of graph centrality measures across multiple runs, as well as by evaluating layer-wise model similarity.

The goal of this project is to explore the structures of the learned graphs from the perspective of network analysis and machine learning. In this manner, ~~we aim to evaluate the interpretability and reliability of the learned graphs~~ and to determine whether further exploration of structural graph properties in EEG-based GNNs is a promising direction for future research.

2 Background

In this section, we introduce key terms and concepts, utilised in our experiments. We cover additional knowledge, needed for understanding the applied methods.

2.1 EEG

EEG is a non-invasive brain monitoring technique used to record electrical activity generated by neurons firing in the brain. The recording is usually executed via metal electrodes (such as silver, gold, or tin) placed on the scalp. EEG provides excellent temporal resolution – it can detect electrical changes in the brain on the order of milliseconds. This makes it appropriate for tracking rapid changes in brain dynamics. The recorded signals contain brain activity across distinct frequency bands ([?], [?]), each linked to specific cognitive and physiological states:

- **Delta waves** (0.5–4 Hz): Linked to deep sleep.

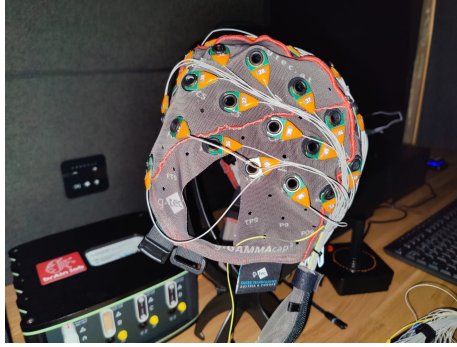


Figure 1: Example EEG capturing device from brAInlab at the IT University of Copenhagen. Seen here are electrodes placed according to the 10-20 system. This capturing device has more electrodes than the one used in this project.

- **Theta waves** (4–8 Hz): Associated with subconscious activities.
- **Alpha waves** (8–13 Hz): During states of relaxed wakefulness and are most pronounced in the further back regions of the head.
- **Beta waves** (13–30 Hz): Linked to active thinking, increased alertness, and anxiety.
- **Gamma waves** (30+ Hz): Indicate information processing and the commencement of voluntary movements.

Recording and working with EEG data poses significant challenges due to high dimensionality and high sensitivity to noise. Multiple electrodes record multiple signals simultaneously, thus resulting in high dimensionality and requiring significant computing power to analyse ([?]). Additionally, EEG signals are highly susceptible to noise and artefacts from both internal sources (such as eye blinks and muscle movements) and external sources (electrical interference), giving them a low signal-to-noise ratio. EEG has limited spatial resolution, making it difficult to pinpoint exact activity sources within the brain. Due to these factors, machine learning techniques become necessary for effective EEG analysis ([?]).

2.2 EEG as a Data Type

The aforementioned high dimensionality and noise sensitivity difficulties are further amplified in motor imagery tasks, where participants imagine movements rather than physically performing them, resulting in subtle neural signals that are harder to detect and classify.

The EEG data used in this study were recorded using 22 Ag/AgCl electrodes placed according to the international 10–20 system, a widely adopted standard that positions electrodes at 10% or 20% intervals between anatomical landmarks such as the nasion (indentation between forehead and nose), inion (rear lower part of the skull), and preauricular points (in front of the ears). This system ensures consistent and reproducible electrode placement across different experiments and provides balanced coverage of cortical areas. The electrodes in our

dataset – including Fz, FCz, Cz, Pz, POz along the midline, and FC3/4, C3/4, C1/2, C5/6, CP3/4, CP1, CPz, P1/2 – are symmetrically distributed across the scalp, odd-numbered electrodes on the left and even on the right side. This layout enables the monitoring of activity across frontal (F), frontocentral (FC), central (C), centroparietal (CP), parietal (P), and parieto-occipital (PO) regions, with a particular focus on sensorimotor areas that are especially relevant for motor imagery and cognitive processing.

2.3 Graph Neural Network

GNNs are a class of deep learning models specifically designed to work with graph-structured data, using specialized convolution operations that can handle non-grid-like structures ([?]).

The operation at the core of GNNs is neural message passing, in which information (referred to as "messages") is exchanged between nodes using neural networks ([?]). At each iteration, the hidden embeddings of each node are updated based on information from their surrounding neighbours ([?]). The process is described by the following expressions:

$$\begin{aligned} m_{N(i)}^{(k)} &= \text{AGGREGATE}^{(k)} \left(\left\{ h_j^{(k-1)} : v_j \in N(v_i) \right\} \right) \\ h_i^{(k)} &= \text{COMBINE}^{(k)} \left(h_i^{(k-1)}, m_{N(i)}^{(k)} \right) \end{aligned}$$

In the above equation, $h_i^{(k)}$ is the embedding of node i at the k iteration, $m_{N(i)}^{(k)}$ is the message aggregated from i 's neighbourhood, and AGGREGATE and COMBINE are predefined differentiable functions ([?]).

At each iteration, the embeddings from each node's neighbourhood are aggregated using a predefined aggregation function. Following that, the aggregated information for each node is combined with the node's previous embedding using a predefined update function to produce the node's new embedding ([?]). Through successive iterations, node embeddings increasingly capture information from progressively larger neighbourhoods, encoding both structural and feature-level patterns ([?]).

GNNs have gained significant attention across various domains and have been systematically benchmarked in prior literature. Notably, the comprehensive survey in [?] evaluated their performance across a range of time series tasks, including classification, forecasting, and imputation, demonstrating their versatility and potential across multiple applications.

In the context of neuroscience, particularly with EEG data, GNN-based classifiers have shown promising results. Studies such as [?] report that GNNs outperform traditional CNNs on EEG classification tasks, while also offering greater interpretability, which is a crucial aspect in biomedical and cognitive research.

2.4 Dynamic Graph Convolutional Neural Network

DGCNN is a specialised type of GNN. It was introduced in [?]. The DGCNN architecture implements the idea of learnable graphs, making GNNs functional

without the need to explicitly provide a graph structure.

Traditional GNNs require a graph structure as an input. This parameter is not always readily available, nor is it always viable to construct one using heuristic methods, due to some data being complex, obscuring the innate graph. Learning-based graphs define edge weights as trainable parameters that are updated during model training, allowing dynamically constructed graph adjacency matrices ([?]). The work of [?] discusses the DGCNN’s potential in encoding EEG data.

Firstly, the trainable parameters of the network are initialised, including the adjacency matrix, with its weights being trainable parameters. Subsequently, the $\text{ReLU}(x) = \max(0, x)$ operation is applied to the adjacency matrix to ensure all weights are positive, upon which the Laplacian matrix is calculated from the adjacency matrix, then normalized. The formula to calculate the Laplacian matrix can be seen in Equation ??.

$$L = D - W \in \mathbb{R}^{N \times N} \quad (1)$$

In the above equation, $W \in \mathbb{R}^{N \times N}$ is the adjacency matrix, and $D \in \mathbb{R}^{N \times N}$ is a diagonal matrix, where each diagonal element is calculated as $D_{ii} = \sum_j w_{ij}$, and all other elements have the value 0.

The obtained Laplacian matrix is normalised using the formula in Equation ??.

$$L' = \frac{2L}{\lambda_{max}} - I_N \quad (2)$$

The Chebychev polynomial items $T_k(L')$, for $k \in \{0, 1, \dots, K - 1\}$, where K is the number of graph convolutional layers. The polynomial items are calculated using the recursive expression in ??

$$T_0(x) = 1 \quad (3)$$

$$T_1(x) = x \quad (4)$$

$$T_k(x) = 2x \cdot T_{k-1}(x) - T_{k-2}(x) \quad (5)$$

The graph convolution operation is then applied using the formula in Equation ??.

$$\sum_{k=0}^{K-1} \theta_k T_k(L')x \quad (6)$$

ReLU is applied to the result of the graph convolution operation, upon which it is input to the upcoming fully connected layer.

Given the DGCNN’s predictions, the loss is calculated using the Cross Entropy loss function. The adjacency matrix and other trainable parameters are then updated using an iterative optimisation algorithm such as gradient descent, this whole process being repeated until the specified convergence criteria is met.

As it can be seen from the DGCNN algorithm, the adjacency matrix W is constructed dynamically, with its weights being randomly initialised, and then updated using gradient descent. The initialisation process introduces randomness and potential instability in the learned graphs, as different initialisations of

the trainable parameters may lead to different local minima during optimisation, and thus different learned parameters, leading to varying graph representations.

From the graph that the network trains, we have a map of how the electrodes communicate according to the network, providing us with a counterpoint to traditional interpretations. Dynamically inferring this relationship leads to significantly better model performance ([?]). Furthermore, having this relationship be determined through learning could yield more insight.

2.5 Similarity Measures

To explore any potential stability in the learned graphs, we turn to similarity measures. These measures are essential tools for quantifying how alike two entities are – an important concept that is deceptively non-trivial. At its core, similarity quantifies the degree to which two objects share features or structure.

Measuring similarity also provides insight into consistency and reproducibility. If a method cannot yield similar outcomes under similar conditions, it becomes difficult to interpret the underlying process or validate the results. In this way, similarity is not just a tool for comparison, but a metric for reliability.

In our work, we apply a kernel-based similarity metric called centred kernel alignment (CKA), which allows us to compare internal representations of neural networks.

2.6 Centred Kernel Alignment

Originally introduced in [?] for comparing representations in deep learning, CKA is invariant to isotropic scaling, linear transformations, and orthogonal transformations, making it a robust metric for similarity ([?]). A robust similarity is needed in this project for two main reasons: it is a powerful metric, giving us a high-level view of it, and secondly, that it gives us an outside perspective to the graph measures.

In this project, we use linear CKA, which begins by computing centred Gram matrices for the activations of each model. These matrices capture pairwise similarities between data points in the representation space. The CKA score is then computed using a normalised version of the Hilbert-Schmidt Independence Criterion (HSIC), resulting in a single similarity value between the two models' representations. Equation ?? shows how a single CKA is calculated.

$$CKA(K, L) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K)HSIC(L, L)}} \quad (7)$$

To evaluate similarity across multiple models, we compute a full CKA matrix. The CKA computes one score between two specific layers, A full CKA matrix is then the scores between all layers. Each row and column corresponds to a specific layer in a specific model. Diagonal entries of this matrix represent comparisons between the same layers across models and thus reflect their consistency under different initialisations.

2.7 Graph Measures

To infer graph stability, we have to understand the graph properties. These require certain qualities to be met – mainly, that they are not isomorphic and that they are connected. If two graphs are isomorphic, it means they have exactly the same shape, regardless of labels. A connected graph is a graph where all nodes are connected to the same graph, i.e. there are no components. Our graphs need to be connected because otherwise, centrality loses meaning, as most of them evaluate walks, and no walk exists between components.

To investigate whether the DGCNN models capture meaningful functional relationships, we analysed the most prominent nodes in the learned graphs using a range of graph centrality measures. This approach also enabled us to compare the structural properties of the learned graphs across multiple training runs. The centrality metrics considered in this analysis include betweenness centrality, barycentres, Laplacian centrality, and current-flow betweenness centrality.

2.7.1 Betweenness Centrality

Betweenness centrality measures how often a node lies on the shortest paths between other nodes. Seen in Equation ??.

$$c_B(v) = \sum_{s,t \in V} \frac{\sigma(s,t|v)}{\sigma(s,t)} \quad (8)$$

where $\sigma(s,t)$ is the number of (s,t) shortest paths, and $\sigma(s,t|v)$ is the number of shortest paths passing through the node v .

The betweenness centrality score reflects a node’s importance for the information flow in a network - how critical the node is in connecting different parts of the network. Betweenness is an often-used metric, with its easy implementation and recognition, making it a common choice for analysing network structure.

2.7.2 Current-flow Betweenness Centrality (Random-Walk Betweenness)

While betweenness centrality gives us insight into the most critical bridges in terms of shortest routes, real-world networks often operate through more diffuse pathways. To capture this, we turned to a more nuanced metric, current flow betweenness centrality, also known as random-walk betweenness. While betweenness centrality only considers the shortest paths, current-flow betweenness looks at all paths. It is based on random walks between node pairs and evaluates how often a given node will fall on a random walk between another pair of nodes ([?]). The current-flow betweenness centrality scores provide a more robust view of node importance in a network.

2.7.3 Laplacian Centrality

Given that the DGCNN architecture itself is built upon Laplacian matrices ([?]), on this basis we consider Laplacian centrality as a means to evaluate node importance within this framework. As the DGCNN and Laplacian centrality share the same fundamental concept, it may be suitable to use this graph centrality

measure. The Laplacian centrality works by summing the Laplacian energy, and the central node is the one with the highest Laplacian energy ([?]). The Laplacian energy, $E_L(G)$ is defined in Equation ??, where λ_i is the eigenvalue of the network Laplacian matrix. In [?], they describe how the energy is found from a mix of local factors and global characteristics, giving it a balanced approach while prioritising the local neighbourhood.

$$E_L(G) = \sum_{i=1}^n \lambda_i^2 \quad (9)$$

2.7.4 Barycentre

Beyond centrality, we also examine the core of the graph. The barycentre, or graph mean, shifts our focus from importance to position, identifying the node that lies closest, on average, to all others.. It identifies one or more nodes in a graph with the minimal total distance to all other nodes, i.e. shortest distance to all other nodes. That is, it minimises the function described in Equation ??, where $d(v, u)$ is the summed weight between v and u ([?]).

$$\sum_{u \in V(G)} d_G(u, v) \quad (10)$$

We utilise the barycentre to find the centre of the graph representation, and compare to other barycentres to attempt to glean structural and representational similarities.

3 Methodology

In this project, we construct and analyse graph representations of EEG signals using the DGCNN architecture. We work with a benchmark motor imagery dataset, preprocess it, train a GNN, and finally evaluate the structural properties of the learned graphs. The following sections walk through each step of this process.

3.1 Dataset

The data, which we are working with for the duration of our project, comes from the BNCI 2014-001 motor imagery dataset from the MOABB benchmark library ([?]). It consists of 5184 trials of EEG data, collected from 9 subjects. The task assigned to the subjects whose brain activity was documented prompted them to perform different motor imagery tasks. The participants were asked to perform four different tasks, imagining movement of their feet, left hand, right hand or tongue.

The dataset was originally released in [?] as a part of the BCI (Brain Computer Interface) competition IV, aiming to provide high quality data openly accessible to the scientific community. This dataset is commonly used as a benchmark, being present in various publications, as well as the Python EEG benchmark library MOABB ([?]).

This dataset provides us with a working domain that is well explored and a type

of task that is well understood allowing us to focus on our primary concern, graph stability.

3.2 Data preprocessing

As the dataset is well-known, several preprocessing steps are standard for ensuring meaningful downstream learning. EEG data presents challenges due to its high dimensionality and low signal-to-noise ratio. To facilitate model training and improve learning efficiency, we applied a frequency-band decomposition based on the five canonical EEG frequency bands: delta, theta, alpha, beta, and gamma ([?]).

In order to apply this transformation, we used the `BandDifferentialEntropy` transform from the `torcheeg` Python library ([?]). The function first filters the input EEG data into the specified sub-bands. The differential entropy value is calculated for each sub-band using Equation ??:

$$H(x) = \frac{1}{2} \log_2(2\pi e \sigma^2) \quad (11)$$

Where σ^2 is the variance of the signal within a given sub-band. The result is a new feature matrix of the shape `samples × channels × bands`. This matrix, in turn, serves as input for the training of our artificial neural network model.

3.3 Model Training

We employed the DGCNN implementation from the `torcheeg` library ([?]), with training and model architecture hyperparameters summarised in Table ??.

The `torcheeg` DGCNN architecture consists of batch normalisation, a configurable amount of graph convolutional layers, and two fully connected layers. The weights of the adjacency matrix used in the convolutional layers are trainable parameters which are randomly initialised from a Xavier normal distribution ([?]). The ReLU activation function is then applied to ensure all the weights are positive, upon which the values are normalised. During training, the adjacency matrix is dynamically updated, enabling the model to learn an optimised graph structure that reflects the relational patterns in the input data.

Parameter	Value
learning rate	1e-4
epochs	200
weight decay	1e-3

(a) Training parameters

Parameter	Value
input channels	5
electrodes	22
graph convolution layers	2

(b) Model architecture parameters

Table 1: Overview of selected hyperparameters. (a) Training parameters: learning rate, number of epochs, and weight decay. (b) Model architecture parameters: number of input channels, number of electrodes, and number of graph convolutional layers.

Model training was conducted using the Adam optimiser ([?]), with the Cross Entropy loss function as the optimisation objective. A relatively small learning

rate was selected to enhance training stability, while weight decay was incorporated to mitigate overfitting. The number of training epochs was chosen to ensure sufficient learning progression without underfitting or excessive overfitting of the model.

We train models with the specified parameters, several values for the number of hidden channels (see Table ??), for 84 different seeds. In total, we train 756 models, which we use to compare learned graph representations across different random initialisations of the trainable parameters.

8, 16, 24, 32, 40, 48, 56, 64, 128

Table 2: Hidden channel values for model training

For model validation, we are using an 80-20 train-test split. A dedicated validation set was not used to maximise the number of available training samples. Given that the primary objective of this study is not to optimize predictive performance on unseen data, but rather to analyse the structural properties of the learned graph representations, we consider the trade-off of sacrificing a more precise estimate of generalisation performance in favour of a larger training set, to be justified and appropriate for our experiment.

3.4 Graph Quality Checks

With model performance at a suitable level, we need to ensure the validity and applicability of the graph metrics employed in our analysis, we conducted a series of checks on all extracted graphs.

For all graphs, we use the `NetworkX` ([?]) function `is_connected`, which verifies that each graph is connected. Connectivity is a necessary condition for computing various centrality measures, and graphs failing this criterion were excluded from further analysis.

Additionally, we use the `vf2pp.is_isomorphic` function from the same library to check if any two graphs are isomorphic. No isomorphic pairs were identified, highlighting the instability of the learned graphs.

Finally, we removed all the very weak connections with a weight of 0.2 or lower from the adjacency matrix by setting their weight to zero. This serves to reduce the noise from the training and to make the graph markedly simpler to traverse.

3.5 Centrality measures

Once we have ensured that all our graphs are connected and no two graphs are the same, we can inquire about the structure of the graphs. To evaluate and compare the learned graphs, produced by the DGCNN models, we employed several graph centrality metrics. These measures are used to quantify the importance and influence of nodes within a graph, providing an insight into the graphs’ structural properties and potentially helping us identify recurring patterns across them.

3.5.1 Barycentres

The first question we considered was whether to focus on the centre or the periphery of the graphs. Assuming that peripheral nodes are more prone to instability, we directed our attention toward centrality measures, with the hypothesis that core graph structures are more likely to exhibit stability. Using the `NetworkX` barycentre function, we compute the barycentres of the graphs derived from the adjacency matrices of trained DGCNN models. The results are aggregated into a dictionary, organized by model instance. For each model object, we record the barycentre nodes and compute their frequency across all seeds and runs. This produces a distribution highlighting which nodes most consistently emerge as barycentres throughout the experiments. To provide a clear overview, the counts are visualised in bar plots, where each bar represents a node (i.e., electrode), and the height corresponds to the number of times it was identified as a barycentre.

3.5.2 Other centrality metrics

We observed evidence that certain nodes appear more frequently than others, suggesting potential structural patterns in the learned graphs. However, since different centrality measures emphasise distinct aspects of node importance, we decided to expand our analysis beyond barycentres. Specifically, we included three additional centrality metrics: Laplacian centrality, betweenness centrality, and current-flow betweenness centrality. We use the following `NetworkX` functions for each respective metric – `laplacian_centrality`, `betweenness_centrality`, and `current_flow_betweenness_centrality`.

The implementation for each measure follows a consistent approach. For each trained model, we extract the adjacency matrix using a helper function and convert it to a graph object. Afterwards, the centrality values are computed using the appropriate `NetworkX` function and the nodes with the highest scores are stored with the purpose of further analysis. This process is repeated for each metric, and the output values allow us to investigate how centrality distributions vary across different DGCNN configurations.

3.6 Use of Graph Measures

With these four graph measures, we obtain a centrality profile for every constructed graph. Rather than analysing each graph individually, we focus on the centrality metrics themselves, which allows for a more comprehensive comparison across models. To extract meaningful insights, we aggregate the centrality results across all model instances and visualise them using bar charts. These visualisations display how often specific nodes (i.e., electrodes) are identified as central, such as barycentres or Laplacian centres. This highlights patterns in node prominence across different centrality definitions.

This aggregation allows us to apply well-established statistical tools, such as the chi-squared test, to see if the training changed the centre frequencies and if they differ from a uniform distribution. When specific nodes or regions appear frequently as centres, we further investigate their likelihood of being selected, providing a data-driven basis for identifying stable or functionally important nodes in the EEG-based graphs.

3.7 Chi Squared Tests

As we have compiled and aggregated data on the four centrality measures, we need to assess whether training the DGCNN model significantly altered the distribution of graph barycentres, we used Pearson’s Chi squared test.

The test assesses the null hypothesis that the barycentre distribution of the untrained model graphs is the same as the one of the trained model graphs. To ensure a fair comparison, both the trained and untrained models shared identical architectures and random initialisations. The test statistic is defined in Equation ??.

$$\chi^2 = \sum_i^K \frac{(O_i - E_i)^2}{E_i} \quad (12)$$

Where the observed values, O_i , are the counts of each electrode being a barycentre in a graph learned by the trained models, and the expected values, E_i are the counts of each electrode being a barycentre in a graph initialised by the untrained models.

Secondly, we wanted to assess whether the barycentre distribution of the trained model graphs deviates significantly from a uniform distribution, where each electrode has the same probability of being a barycentre.

To assess the hypothesis that the barycentres are uniformly distributed, we conducted another Pearson’s Chi squared test, applying the formula in Equation ?? to calculate the test statistic. The observed values, O_i , are the barycentre counts for the trained model graphs, and the expected values, E_i , are $\frac{1}{K} = \frac{1}{22}$ for all classes, since we are testing against a uniform distribution, where the probabilities of all classes are equal.

3.8 Centred Kernel Alignment Implementation

We apply the CKA metric ([?]) to quantify representational similarity between neural network layers, adapting the PyTorch-based implementation from ([?]). Our version emphasises transparency, readability and computational efficiency, ideal for small-scale datasets. Key modifications to the original approach:

- **Single-Epoch Computation:** CKA is computed only at the final training epoch to reduce runtime and ensure that the analysis reflects the model’s final representational state.
- **Fixed-Sample Evaluation:** Evaluation is restricted to the first $N = 500$ data points, balancing memory usage with statistical reliability.
- **Simplified Hook Management:** Gram matrices are computed on demand using manually registered hooks, minimising use complexity.

Advantage	Limitation
Fixed memory usage	Potential sampling bias
Deterministic results	Higher variance vs. multi-epoch
Simplified debugging	Less scalable

Table 3: Design Trade-offs

3.8.1 Computational Workflow

The CKA analysis pipeline consists of three stages:

1. Model Pair Selection: Comparing every model m_i to every other m_j exactly once

2. CKA Calculation: For each pair of layers in the models', we compute Gram matrices and normalise the HSIC:

```
K = gram_matrix(X.flatten(start_dim=1))
L = gram_matrix(Y.flatten(start_dim=1))
cka_matrix[i, j] = hsic(K, L) / (hsic(K, K).sqrt() * hsic(L, L).sqrt())
```

3. Aggregation: Average CKA

$$\bar{\mathbf{M}} = \frac{1}{n} \sum_{i=1}^n \mathbf{M}_i \quad (13)$$

$$(14)$$

These three stages provide us with the average CKA.

3.9 Likelihood

Now that we have CKA scores and graph measurements, we need to quantify how likely each electrode is to be selected as a centre by our various centrality measures. We introduce a likelihood metric. This measure evaluates the frequency with which each label is chosen as the central node across all samples.

The likelihood is calculated using Equation ???. Specifically, we compute the total likelihood percentage for each label, representing the proportion of times a label is chosen as a centre and the excess likelihood over uniform, which quantifies how much the frequency differs for a label that is selected compared to what would be expected under a uniform distribution.

$$\text{Total Likelihood \% } L_{label}^t = \frac{x_{label}}{x_{total}} \cdot 100 \quad (15)$$

$$\text{Above uniform \% } L_{label}^u = \left(\frac{x_{label}/x_{total}}{1/n_{labels}} \right) \cdot 100 - 100 \quad (16)$$

Here, x_{label} is the number of times a specific label is selected, x_{total} is the total number of centre selections across all labels, and n_{labels} is the number of labels (22 in our case).

4 Results

Following the training of DGCNN models on the BNCI 2014-001 Motor Imagery dataset, we present model performance, our analysis of the resulting graph structures, and internal representations. Rather than focusing on classification accuracy, our primary aim is to investigate the structural patterns and representational consistency that emerge across models trained with different seeds and hidden channel configurations.

To assess the structural properties of the learned graphs, we compute a range of centrality metrics-including barycentres, Laplacian centrality, betweenness centrality, and current-flow betweenness centrality. These metrics allow us to identify nodes that frequently emerge as central or influential across training runs, providing insight into potential functional hubs in the EEG signal graph. In addition to structural analysis, we use CKA to quantify the similarity between internal representations learned by different models. CKA offers a robust measure for comparing the representations across layers and model instances, helping us understand the degree to which different DGCNNs converge on similar solutions despite differing initial conditions.

4.1 Model Performance

In total, we trained 756 GNNs across 84 random seeds and 9 different values for the number of hidden channels. These models achieved an average test accuracy of $48.9 \pm 1.2\%$, significantly outperforming the random-chance baseline of 25%. While model performance is not the central focus of this study, these results confirm that the networks have learned meaningful representations from the data, justifying further analysis of their internal graph structures and learned features.

4.2 Graph Representations

To visualize the structure of the graphs generated by the trained models, we present a representative example in Figure ???. This graph was produced by a model trained with seed 25243 and 24 hidden channels. While no two graphs are identical, this example illustrates the typical structure observed in our models. In the figure, red lines indicate strong edges with weights above 0.7, and the purple node marks the barycentre.

4.3 Average CKA

Figure ?? presents the average CKA similarity across layers for all model pairs from the first 10 seeds. The diagonal of the matrix shows high similarity across corresponding layers, with the top three layers showing similarity scores above 0.9. The final two layers also exhibit strong consistency, with average similarity scores of 0.8 and 0.76, respectively.

4.4 Barycentre Bar Charts

For each learned graph representation, we kept track of the barycentre nodes. We then made a bar chart of the barycentre frequency for all the models' learned

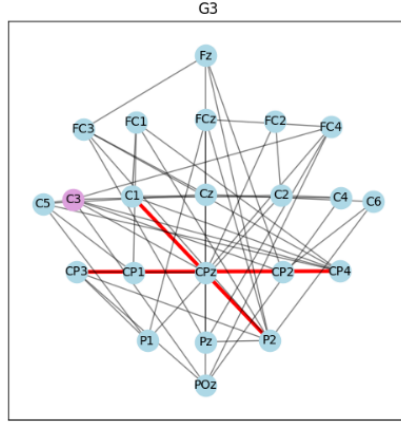


Figure 2: Graph created by seed 25243 with 24 hidden channels; The purple node "C3" is highlighted as it is the identified barycentre of the graph; The red edges are the ones, considered strongest connections

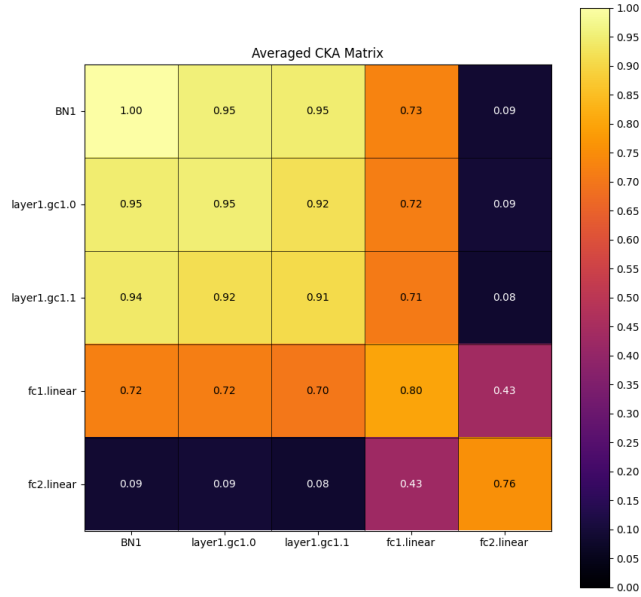


Figure 3: Average CKA for between the 90 models in the first 10 seeds

graphs (Figure ??).

From the plot, we can clearly see that C3, CP3 and CP4 are more commonly the barycentre than all of the other nodes. We also looked at the barycentre bar chart for the untrained models with the same seeds and parameters (Figure ??), to confirm that a difference occurs during training. For the untrained models, the barycentre counts are very similar for all the electrodes, as opposed to the trained model barycentre counts, in which we can clearly see that C3, CP3 and CP4 are more frequent.

To validate whether the difference between the barycentre counts of the trained and untrained models is statistically significant, we performed a Chi squared goodness of fit test. We obtain a χ^2 test statistic of 212, which is higher than the critical value for significance level $\alpha = 0.01$ and $K - 1 = 21$ degrees of freedom, since we have 22 different electrodes. We thus reject the null hypothesis at a 1% significance level, concluding that there is a significant difference in the barycentre counts before and after training the models.

Furthermore, we compared the barycentre counts of the trained models' learned graph representations against the expected values of said counts under a uniform distribution. We have thus performed another Chi squared test to ensure there is a significant difference in the electrodes' probabilities of being barycentres. We obtain a χ^2 test statistic of 172, which exceeds the critical value for significance level $\alpha = 0.01$ with 21 degrees of freedom. We therefore reject the null hypothesis at the 1% significance level, concluding that the electrodes don't have equal probabilities of being barycentres.

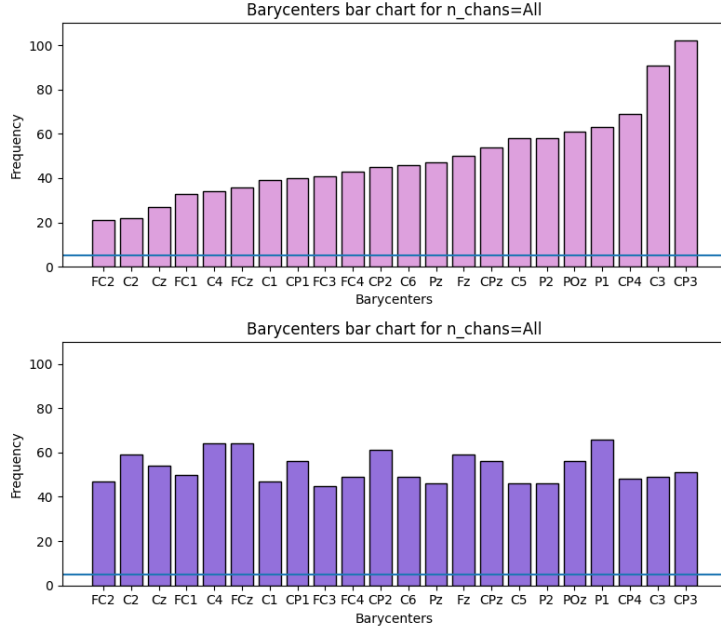


Figure 4: Barycentres bar charts for the trained (top) and untrained (bottom) models; Order of untrained bars is based on the sorted ascending order of the trained results for comparison's sake; The blue line at the frequency of 5 marks the minimal frequency requirement for the execution of a χ^2 test

Since the ordering of nodes in the bar chart is arbitrarily chosen, a more meaningful way to visualise node-level results is by using the MNE library to generate a topomap, a heat map of electrode activity projected onto a head model, as shown in Figure ?? ([?], [?]). In the figure, we see the same information as in Figure ??, but now contextualised on the head. The deep red region on the lower left corresponds to the CP3 and C3 electrodes, indicating a high frequency of barycentre occurrences at that location.

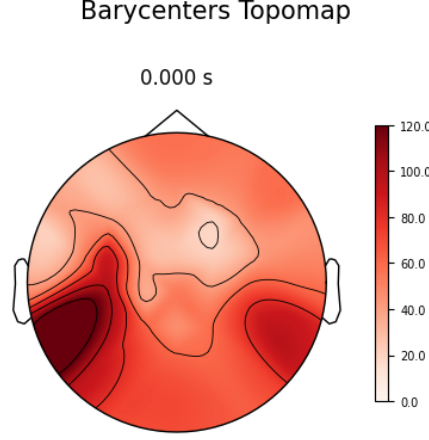


Figure 5: Topological heat map showing barycentre node frequencies. Deeper red equals more centre occurrences

The bar charts for betweenness, current flow and Laplacian centre look similar, with C3, CP3 and CP4 as the highest observed frequencies.

4.5 The Likelihood for Node Barycentres

To quantify the significance of barycentre selection, we compute the likelihood of all nodes being selected as barycentres across all models.

Table ?? shows both types of likelihood we compute: the total likelihood (frequency) and the likelihood of a node being the centre, relative to a uniform distribution. It shows the two most frequently observed barycentres, C3 and CP3. We have showcased the two most likely nodes to occur across all centrality measures; the other 40 likelihoods can be found in Appendix ??

We observe a clear distinction between trained and untrained models: in the trained models, C3 and CP3 are selected as barycentres in approximately 9% of cases, which is 90% more than the uniform. In contrast, the untrained models show no strong preference, with frequencies near 4.5% and low differences from uniform expectations.

These findings suggest that model training leads to the emergence of consistent structural patterns in the learned graph, particularly emphasising the C3 and CP3 electrodes, which are disproportionally selected as barycentres relative to a uniform distribution.

Likelihood:	Total	Above Uniform
C3		
Trained	8.77%	92.94%
Untrained	4.74%	4.28%

(a) C3 Likelihood

Likelihood:	Total	Above Uniform
CP3		
Trained	8.97%	92.34%
Untrained	4.64%	2.08%

(b) CP3 Likelihood

Table 4: The Likelihood of selecting C3 or CP3 as the barycentre, showing total frequency and the likelihood exceeding a uniform distribution.

5 Discussion

The focus of our project is to investigate potential similarities and stability within the learned graphs produced by our model. These graph representations aim to capture which nodes become salient during training, potentially revealing shared patterns that arise from learning the same underlying task across different initialisations.

Before delving into the structure and consistency of these learned graphs, it is essential to first establish that our models are functionally valid. That is, that they perform well enough to justify further analysis. While an average test accuracy of 48.9% may not be remarkable in absolute terms, it exceeds the 25% baseline expected from random guessing in a 4 class classification task. This suggests that the models are learning non-trivial and task relevant features, providing a solid foundation for investigation.

Importantly, our goal is not to maximize classification performance but to understand what the models are learning and whether those learned representations exhibit shared structure or stability across different training seeds. The doubled performance score relative to a random model is sufficient to support a discussion of internal graph representations, enabling us to explore how similar or different the resulting graphs are, and whether certain nodes or patterns consistently emerge as important across model instances.

With this baseline established, the remainder of this discussion focuses on ~~how we quantify representation~~ similarity using CKA, how we interpret the stability of graph metrics such as barycentre, and what these results reveal about the relationship between model learning dynamics and graph structures.

5.1 CKA Results

When using CKA, we assess how similar the internal activations of two models are, layer by layer. This provides a high-level abstraction view of representational similarity without requiring exact alignment between individual neurons. CKA is useful for comparing models trained on the same task but with different random initialisations.

In our analysis, the diagonal values of the CKA matrices are of primary interest. These compare the same layer across different models. High diagonal values indicate that different models tend to learn similar representations at corresponding depths, suggesting a degree of stability and shared internal structure. The off-diagonal values, which compare layers at different depths, are less relevant here, as our focus is on layer wise consistency rather than cross-layer relationships.

On average, the top three layers yield CKA scores of over 0.9, while the final two remain relatively high, around 0.8. This consistency supports the assumption that the underlying learned graphs are likely to exhibit structural or functional similarities, though not exact equivalence.

~~The CKA requires data to be passed through the model in order to compute representation similarity. We use the test dataset for this purpose, as it offers a more reliable evaluation of model performance. One important note about our CKA implementation is that it includes a slight sampling bias: to ensure~~

consistent and reasonable run times, we only evaluate the first half of the test dataset (500 datapoints).

5.2 CKA and Graph Metrics in Relation to Similarity

While CKA provides a measure of representational similarity between models, it does not directly reflect the underlying graph structures produced by the DGCNN. Instead, CKA evaluates how similarly two models organise information in their internal activations, independent of specific neuron or node alignments. We performed CKA comparisons across the first 10 seeds, resulting in 90 pairwise similarity matrices. These were averaged into a single CKA matrix, which also showed high values along the diagonal, indicating that different model instances tend to learn similarly structured representations at corresponding layers. However, this similarity in activations does not necessarily imply structural similarity at the graph level, such as consistent importance of specific nodes or edges. To investigate that aspect, we turn to graph metrics such as graph centrality measures.

5.3 Graph measures and expectations

To evaluate the stability of the extracted graph representations, we computed pairwise graph isomorphism among all graphs derived from the trained models. No two graphs were found to be isomorphic, indicating that each model produces a structurally distinct graph. This suggests a degree of instability in the learned representations.

Upon reflection, we recognise that while graph isomorphism is a sufficient condition to establish structural uniqueness, it is lacking. Isomorphism evaluates structural similarity while ignoring node labels. However, in our case, node labels correspond to EEG electrodes, and thus carry meaningful spatial and physiological significance. As a result, two graphs that are structurally identical but differ in node labelling would be functionally distinct.

That said, this distinction does not affect our conclusions. Since none of the graphs are isomorphic, and isomorphism allows for node label permutation, it follows that none of the adjacency matrices are identical either.

When calculating graph centrality metrics on the extracted graphs, we hypothesised that certain nodes would consistently emerge as central across models. This expectation was grounded in the fact that all models shared identical architectures and training dataset, with the main source of variability being the random initialisation of the trainable parameters.

Our results largely support this hypothesis, with the electrodes C3, CP4, and CP3 emerging among the most frequent across all examined graph centrality measures. This includes barycentres, Laplacian centrality, betweenness centrality and current-flow betweenness centrality.

Furthermore, comparisons between the trained and untrained model graphs show a clear shift in the central nodes. While in the untrained model graphs the distribution of barycentre counts is very similar to a uniform distribution, the trained model graphs show considerably higher counts for electrodes C3 and CP3 than for the other electrodes. These counts show significant increase from

their expected values if the distribution was uniform. This difference is also highlighted by our Pearson’s Chi squared test, which shows significant deviations from a uniform distribution at the 1% significance level.

5.4 Likelihood analysis

The consistent emergence of C3 and CP3 as hubs across all centrality measures reveals properties of how DGCNNs process EEG data. These nodes exhibited a 9% likelihood of being identified as centres, representing a 90% increase over the uniform distribution baseline. This pattern stands in stark contrast to untrained models, which showed no meaningful structure with 4.5% centrality likelihood and less than 5% deviation from uniform, demonstrating that the trained networks develop organised representations rather than random connectivity patterns.

5.5 Hidden channel bar chart counts and chi-squared analysis

Given that we have found C3, CP3 and CP4 as the most common barycentres, we expected to see these electrodes as some of the most frequent nodes for other centrality measures as well. We observed that for all of the centrality measures that we have experimented with, namely barycentres, betweenness centrality, Laplacian centrality and current-flow betweenness centrality, C3 and CP3 consistently appear as the two most frequent electrodes, followed by CP4.

To further explore these patterns, we analysed centrality measure distributions grouped by the number of hidden channels in the corresponding model architectures. We examined bar charts grouped by each number of hidden channels listed in Table ??, resulting in nine bar charts per centrality metric. Across all hidden channel configurations, C3, CP3, and CP4 remain consistently prominent, reinforcing the hypothesis that certain electrodes hold significance in the learned representations. Interestingly, the prominence of these electrodes appears to be clearer in models with a higher number of hidden channels, which could suggest that increased model capacity may amplify structural consistency. However, since the number of samples for each grouping is limited, further scaling of the experiment would be necessary to reach a conclusion. Additionally, we cannot validate these findings using another Pearson’s Chi squared test due to some electrodes having a count lower than 5, which is the minimum count required for this statistical test. Increasing the number of samples or using an alternative statistical test could therefore be a consideration in future research.

5.6 Issues with the benchmarking dataset

In this project, we utilised the BNCI 2014-001 Motor Imagery dataset as the basis for training and evaluating the DGCNN models. The aforementioned dataset is widely adopted in the literature and serves as a common benchmark for EEG Motor Imagery tasks, making it a practical choice for our study. Despite this fact, the dataset presents certain limitations, such as it being a relatively old dataset, as well as the relative simplicity of the task.

The experimental scope of this study was limited to a single dataset and a single

model architecture, the DGCNN implementation provided by the `torcheeg` Python library. Despite the promising results of our study, an experiment at a bigger scale, comprising more datasets and more EEG tasks rather than just Motor Imagery would be required to assess whether learning consistent graph central nodes is a feature of this type of deep learning models. Only using this simple dataset has allowed us to focus on analysing graph stability, showcasing promising results which can be further explored and validated in future studies.

5.7 Physiological Alignment of Salient Nodes

Another part of our analysis involves identifying nodes that consistently appear as central across multiple centrality measures in the learned graph structures. The electrodes C3, CP3, and CP4 consistently show the highest scores across all graphs in the Laplacian, betweenness, and current-flow betweenness centrality measures and in barycentre analyses. These measures were selected because they effectively capture different aspects of node importance in a graph. We hypothesise that the brain regions corresponding to these electrodes, being most active during motor imagery tasks, produce distinctive EEG patterns, which may lead to these nodes consistently appearing as central in the learned representations. This consistency suggests a stable and dominant role in the structure of the learned representations.

According to the international 10-20 EEG electrode placement system and established neuroscientific literature, C3, CP3 and CP4 are situated over or near the motor cortex. This region of the brain is responsible for planning and executing voluntary movements. Specifically, C3 is located over the left motor cortex, which is the part of the brain responsible for controlling movement on the right side of the body. CP3 and CP4 are positioned just behind this region, over areas known as the sensorimotor cortex, whose function is coordinating movement and processing sensory information.

This is noteworthy in the context of our dataset, where participants were asked to imagine movements of their hands, feet, or tongue. The fact that these electrodes consistently emerge as central nodes in our learned graphs suggests that the DGCNN model instances are not only learning task-relevant features but are also capturing physiologically meaningful structures. This provides credibility to the model’s internal representations, as they appear to align with known brain anatomy. Moreover, the repeated emergence of the central-region nodes across different model initialisations further supports the stability of the learned representations. This consistency suggests that the networks are capturing reliable and interpretable patterns related to the motor imagery tasks. These findings point toward a connection between abstract model behaviour and concrete neurophysiological understanding, aiding both the interpretability and the trustworthiness of graph deep learning models in neuroimaging applications.

6 Conclusion

We trained DGCNN models on the BNCI 2014-001 Motor Imagery dataset and analysed the resulting learned graph representations. The primary objective was to answer our research question of whether the learned graph representa-

tions exhibit any form of underlying stability across runs. For this purpose we employed both model similarity analysis using CKA and a range of graph centrality measures.

Our findings reveal a pattern: the electrodes C3 and CP3 consistently emerged as the central nodes across several centrality metrics, namely barycentres, betweenness centrality, Laplacian centrality, and current-flow betweenness centrality. Thus, for our particular experiment, we observe a certain degree of consistency in which electrodes have high importance scores in the learned graphs according to the aforementioned centrality measures.

Given that these electrodes are located over the motor cortex, area involved in motor planning and execution, this finding suggests that the models are learning structured graph representations, and are also capturing meaningful and interpretable neural features specific to the motor imagery task.

While these results are promising, the study was limited to a single dataset and a single GNN architecture. Broader validation across multiple EEG datasets and diverse neural network configurations is necessary to determine the generalizability of these findings. Nevertheless, the observed emergence of biologically related graph structures highlights the potential of GNNs for learning interpretable and stable representations in EEG analysis, providing motivation for future research in this area.

7 Future Work

8 Bibliography

[heading = none]

A Additional Bar charts

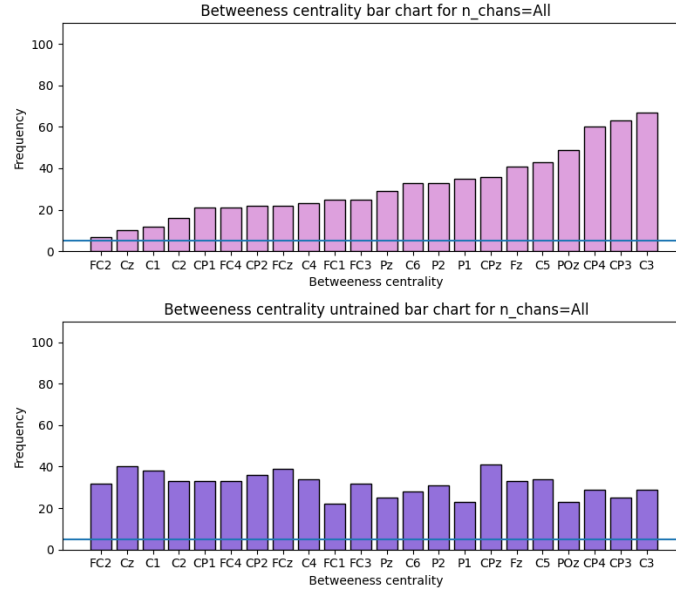


Figure 6: Betweenness centrality bar charts for the trained (top) and untrained (bottom) models; Order of untrained bars is based on the sorted ascending order of the trained results for comparison's sake; The blue line at the frequency of 5 marks the minimal frequency requirement for the execution of a χ^2 test

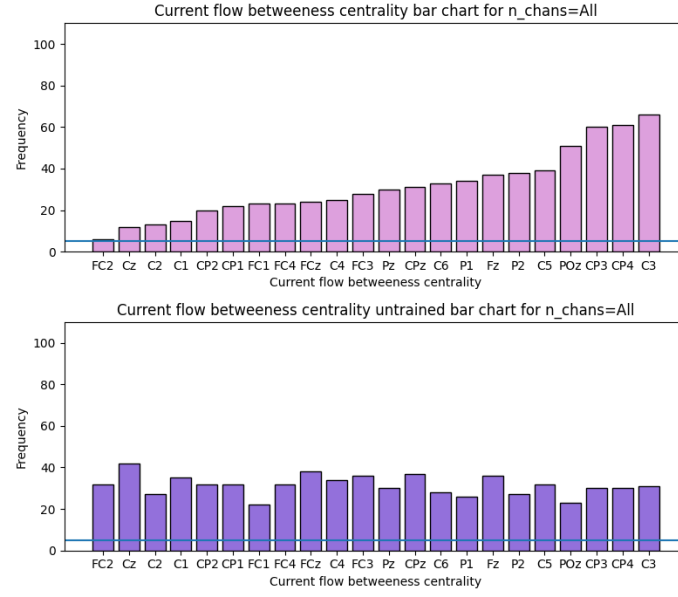


Figure 7: Current-flow betweenness centrality bar charts for the trained (top) and untrained (bottom) models; Order of untrained bars is based on the sorted ascending order of the trained results for comparison's sake; The blue line at the frequency of 5 marks the minimal frequency requirement for the execution of a χ^2 test

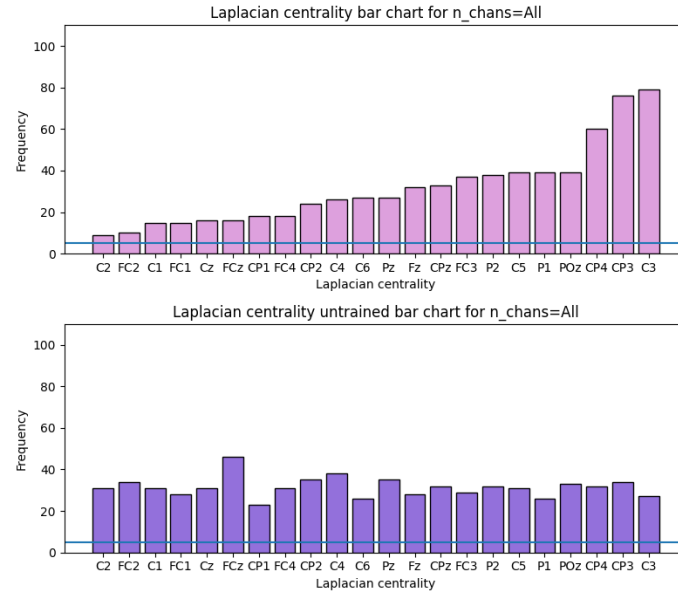


Figure 8: Laplacian centrality bar charts for the trained (top) and untrained (bottom) models; Order of untrained bars is based on the sorted ascending order of the trained results for comparison's sake; The blue line at the frequency of 5 marks the minimal frequency requirement for the execution of a χ^2 test

B Full Likelihood table

Node Label	Total Trained	Likelihood Untrained	Above Uniform Trained	Likelihood Untrained
CP3	8.97%	97.38%	4.64%	2.02 %
C3	8.77%	92.94%	4.74%	4.23 %
CP4	6.25%	37.50%	4.44%	-2.42 %
POz	6.04%	33.06%	3.73%	-17.94 %
P1	5.75%	26.41%	5.14%	13.1 %
P2	5.54%	21.98%	4.54%	-0.2 %
C5	4.94%	8.67 %	4.44%	-2.42 %
CPz	4.94%	8.67 %	5.75%	26.41 %
Fz	4.94%	8.67 %	4.44%	-2.42 %
CP2	4.33%	-5.64 %	5.14%	13.1 %
C6	4.13%	-9.07 %	4.23%	-6.85 %
Pz	4.13%	-9.07 %	4.44%	-2.42 %
FC3	3.83%	-15.73%	4.54%	-0.2 %
CP1	3.73%	-17.94%	4.33%	-4.64 %
FC4	3.73%	-17.94%	4.94%	8.67 %
C1	3.73%	-17.94%	4.74%	4.23 %
FCz	3.43%	-24.6 %	6.05%	33.06 %
FC1	3.33%	-26.81%	4.03%	-11.29 %
C4	3.02%	-33.47%	5.24%	15.32 %
Cz	2.52%	-44.56%	5.75%	26.41 %
C2	2.12%	-53.43%	5.04%	10.89 %
FC2	1.81%	-60.08%	5.95%	30.85 %

Table 5: Caption

C All Topological maps

Betweenness centrality Topomap

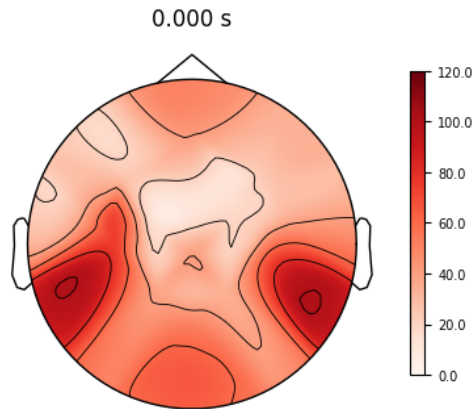


Figure 9: Topomap of betweenness centrality frequencies of nodes; Deeper shade of red corresponds to higher node frequency

Current-flow Betweenness Topomap

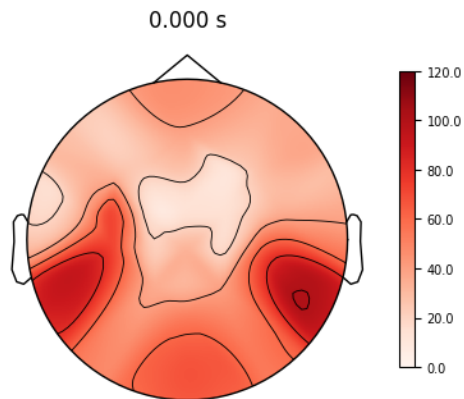


Figure 10: Topomap of current-flow betweenness centrality frequencies of nodes; Deeper shade of red corresponds to higher node frequency

Laplacian Centrality Topomap

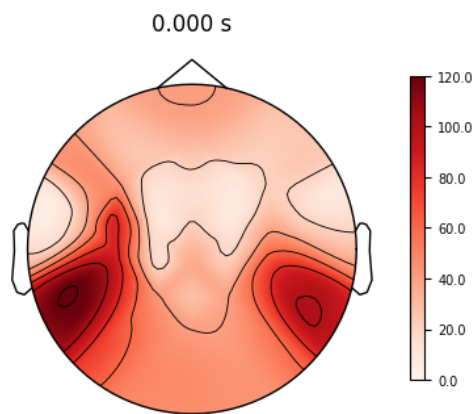


Figure 11: Topomap of Laplacian centrality frequencies of nodes; Deeper shade of red corresponds to higher node frequency